access items within the `Request.Form` collection. This type of approach is common in many web frameworks and should be familiar to classic ASP and PHP developers as well as developers who dabbled with the ASP.NET Web Pages framework. You use `StringValues.IsNullOrEmpty` to determine whether a value exists for the specified key. It will return `false` if no value has been provided or if the specified key doesn't exist in the form collection.

The reason for StringValues

The `StringValues` type was introduced to streamline the handling of values that may be empty, single strings, or multiple strings. It is perfectly possible to assign the same name to multiple controls in a form. You might do this if you want to allow the user to make multiple selections using a series of check boxes, for example. Suppose you want to provide the user with the ability to select multiple cities, and you provide multiple check boxes—each with their name attribute set to `city`. When a selection has been made and the form has been submitted, the selection might look like this as part of the request body:

```
city=London&city=Paris&city=Berlin
```

In previous versions of ASP.NET, `Request.Form` would return multiple values based on the same key as a single comma-separated string, so `Request.Form["city"]` would return `"London,Paris,Berlin"`. It would be your responsibility to convert this string—perhaps using the `string.Split` method to generate an array from it—so the posted values could be iterated and processed individually.

The `StringValues` object represents zero (null), one, or more strings and supports an implicit conversion to `string[]`, which means you can use iteration statements directly in an item in the `Request.Form` (and `Request.Query`) collection without having to manually convert it to a collection of some kind:

```
foreach(var city in Request.Form["city"])
{
    ...
}
```